AN1.11 - Credit Ledger (A9)

Sprint: AT24 AI Agents - Agent Framework Foundation **Step:** A9 - the agent Credit Ledger (real accounting, not a counter) **Depends on:** A3 persistence, A4 runtime, A8 authorization (all closed) **Gate:** G09 - **CLOSED / APPROVED**. Migration **applied under G09 authorization**.

Post-apply verification (live DB)

`prisma migrate deploy` -> *"Applying migration `20260907120000_add_agent_credit_ledger` ... All migrations successfully applied."*

Check	Found
enum `AgentCreditEntryKind`	present
table `AgentCreditLedgerEntry`	present
unique index `AgentCreditLedgerEntry_idempotencyKey_key`	present (UNIQUE)
indexes	`(userId, periodStart)`, `(runId)`, `(userId, createdAt)` + pkey
billing tables (`Billing` / `Subscription` / `Plan`)	intact (3/3)
`_prisma_migrations` row	present, finished
`prisma migrate status`	*"Database schema is up to date"*
`prisma generate`	succeeds
real-persistence smoke (`PrismaCreditStore`)	`charge` -> entry; duplicate key -> `alreadyApplied: true` (idempotent); `amount 200` vs balance `100` -> `InsufficientCreditsError`, **no entry**; `refund 10` -> `-10` entry; `historyForRun` -> 2 entries; `balance` = `100 − 30 + 10 = 80`

The migration file keeps its review-time header (immutable once applied). **No credit wiring beyond the reservation/reconcile already in `tick()` was added.**

> **This is not `run.creditsConsumed += cost`.** That field stays as the per-run *budget* counter feeding A8's `maxCreditCost` guardrail. A9 is a separate **accounting authority**: **Account (allowance) -> immutable ledger -> deterministic balance -> run/tool correlation -> idempotent charge -> auditable history.**

---

1. Budget vs accounting - the separation (owner's point 2)

	mechanism	owner
Budget / resource guardrail	`AgentRun.limits.maxCreditCost` - a per-run ceiling; `checkLimits()` denies a tool call that would cross it	A4 / A8
Entitlement / allowance	`PLAN_LIMITS[planId].aiCredits` for the billing period	`config/plan-limits.ts` (unchanged)

	mechanism	owner
Accounting authority	`AgentCreditLedgerEntry` - immutable entries; `balance = allowance − Σ(entries this period)` recomputed fresh	**A9 (new)**

A8 decides **whether authorized**. A9 decides **whether economically executable**. Neither absorbs the other - the runtime calls A8 `authorize()`, then A9 `charge()` (reservation), then the executor.

---

2. What was built - `services/agent-framework/credits/`

File	Role
`credit-store.ts`	`CreditStore` **port** (`insert` / `findByIdempotencyKey` / `sumForPeriod` / `entriesForRun`) + `DuplicateLedgerEntryError`. The ledger never touches Prisma directly.
`in-memory-credit-store.ts`	`InMemoryCreditStore` - tests + stand-in until the migration is applied.
`prisma-credit-store.ts`	`PrismaCreditStore` - the real `AgentCreditLedgerEntry` table; maps a Postgres unique-violation (`P2002`) to `DuplicateLedgerEntryError`.
`allowance-resolver.ts`	`AllowanceResolver` port + `PlanAllowanceResolver` (reads `User.planId` / active `Subscription` -> `PLAN_LIMITS[plan].aiCredits` + period bounds) + `FixedAllowanceResolver` (tests). **No pricing logic.**
`credit-ledger.ts`	`CreditLedger` - `balance` / `canAfford` / `charge` / `refund` / `historyForRun`; `InsufficientCreditsError`.
`index.ts`	barrel + `createCreditLedger()` (Prisma-backed; `AGENT_CREDIT_INMEMORY=1` escape hatch for harnesses) + `inMemoryCreditLedger()`.

2.1 `CreditLedger` semantics

```
balance(userId)          -> { allowance, consumed, balance }  - allowance − Σ(period entries), NEVER cached
canAfford(userId, n)     -> balance >= n
charge({ ..., amount, idempotencyKey })
  amount < 0                 -> Error (use refund)
  idempotencyKey exists      -> no-op, { alreadyApplied: true }, balance unchanged
  balance < amount           -> InsufficientCreditsError  (deterministic; NO entry written; NO negative bala
  otherwise                  -> insert immutable entry { amount, balanceAfter, kind, runId, stepId?, toolCal
  concurrent duplicate race  -> caught (unique key), treated as alreadyApplied
refund({ ..., amount, idempotencyKey })   -> a signed (negative-amount) entry, kind "refund"
historyForRun(runId)     -> every charge/refund for the run, ordered, fully attributed
```

2.2 Runtime integration - reservation + reconcile (no double-charge on resume)

`doRunningSlice`, per plan step:

```
1. authorizationService.authorize(...)            (A8)  deny -> terminate
2. creditLedger.charge({ amount: estimate,        (A9 RESERVE, BEFORE the executor)
     idempotencyKey: `${runId}:step${planIndex}:reserve` })
     InsufficientCreditsError -> terminate credit_limit / insufficient_credits
3. invokeTool(...)                                (executor)
4. reconcile: actual = ToolResult.creditsConsumed
     actual > estimate -> charge  `${runId}:step${planIndex}:topup`
     actual < estimate -> refund  `${runId}:step${planIndex}:refund`
```

```
     (A2 tools are flat-cost -> delta 0 -> no reconcile)
5. run.creditsConsumed += actual                  (the A8 budget counter, unchanged)
6. advance nextPlanIndex; persist
```

Idempotency keys are derived from `(runId, plan step index)`, never from a fresh `toolCallId`. A tick that dies after the reserve but before advancing `nextPlanIndex` re-runs the same step on resume: the reserve `charge()` hits the existing key -> `alreadyApplied` no-op. **A read tool may re-execute on a mid-tick crash; it is never re-charged.**

---

3. Persistence - `AgentCreditLedgerEntry` (migration GENERATED, NOT APPLIED)

`prisma/schema.prisma`: +1 enum `AgentCreditEntryKind` (`tool_call`, `model_inference`, `research_search`, `backtest`, `optimization`, `large_context`, `refund`, `adjustment`), +1 model:

```
AgentCreditLedgerEntry
 id, userId, runId, stepId?, toolCallId?, kind,
 amount        (signed: + debit, − refund),
 balanceAfter  (advisory - computed at write time; recompute for authority),
 idempotencyKey  @unique  <- the double-charge guard,
 reason, periodStart, createdAt
 @@index([userId, periodStart])  <- the period-sum query
 @@index([runId])                <- historyForRun
 @@index([userId, createdAt])
```

Append-only, immutable - no `updatedAt`, no `deletedAt`, no update path.

Migration `20260907120000_add_agent_credit_ledger/migration.sql` - **GENERATED via `prisma migrate diff` (offline), hand-reviewed, header marks it NOT APPLIED.** Purely additive: 1 enum + 1 table, no `DROP`, no `ALTER TABLE`, never references an existing table.

---

4. G09 proof - `npm run validate:agent-credit` -> 13 passed, 0 failed

balance = allowance – ledger sum, recomputed every call	■
`canAfford` = balance >= amount	■
charge: atomic debit; immutable entry with `balanceAfter` + `runId`/`kind`/`reason`/`toolCallId`	■
IDEMPOTENT: same `idempotencyKey` -> no-op, balance unchanged, **one** entry	■
NO NEGATIVE BALANCE: over-budget charge -> `InsufficientCreditsError`, **no entry written**, balance untouched	■
charge rejects a negative amount (use `refund`)	■
refund: signed negative-amount entry raises the balance	■
historyForRun: every charge/refund, ordered, fully attributed (`kind`, `amount`, `toolCallId`, `reason`)	■
generated `AgentCreditEntryKind` enum == `AF-v1` vocabulary	■
`PlanAllowanceResolver`: unknown user -> `free` plan's `aiCredits` (`PLAN_LIMITS.free.aiCredits`)	■
migration present; `NOT APPLIED`; additive-only; unique `idempotencyKey` index	■

RUNTIME: allowance 1, 2-tool plan -> 1st tool reserves 1 (balance 0), 2nd reservation fails -> run terminates `credit_limit` / **`insufficient_credits`** (A9, distinct from A8); `historyForRun` shows exactly 1 charge	■
RUNTIME: a well-funded run, a lost tick between reserve and advance -> resume charges **exactly once** per tool call (idempotent reservation)	■

4.1 Auditability - given a run, `historyForRun(runId)` answers

what was charged - for which tool (`toolCallId` + `reason`) - when (`createdAt`) - why (`reason`) - how much (`amount`) - was it retried (an `alreadyApplied` no-op leaves one entry) - was it refunded (a negative `kind: "refund"` entry) - resulting balance (`balanceAfter` + a fresh `balance()` recompute agree).

4.2 Regression - no gate lost

`validate:agent-contracts` 38/0 - `validate:agent-tools` 20/0 - `validate:agent-run-persistence` 17/0 - `validate:agent-runtime` 9/0 - `validate:agent-supervisor` 11/0 - `validate:agent-integrity` 21/0 - `validate:agent-memory` 19/0 - `validate:agent-authorization` 17/0 - `validate:agent-credit` **13/0**. (The 4 runtime-exercising harnesses set `AGENT_CREDIT_INMEMORY=1` so they never write real ledger rows.)

---

5. TypeScript

`npx tsc --noEmit`: **0 errors** in the framework code + the new `AgentCreditLedgerEntry` model. Repo-wide 77, all in the stale generated `.next/dev/types/validator.ts` (gitignored, pre-existing).

---

6. Files changed

Added (7):

```
frontend/services/agent-framework/credits/credit-store.ts
frontend/services/agent-framework/credits/in-memory-credit-store.ts
frontend/services/agent-framework/credits/prisma-credit-store.ts
frontend/services/agent-framework/credits/allowance-resolver.ts
frontend/services/agent-framework/credits/credit-ledger.ts
frontend/services/agent-framework/credits/index.ts
frontend/prisma/migrations/20260907120000_add_agent_credit_ledger/migration.sql   (GENERATED + REVIEWED, NOT
frontend/scripts/validate-agent-credit.ts
```

Modified (5):

```
frontend/types/agent-framework/agent-run-contract.ts   + AgentCreditEntryKind + AgentCreditLedgerEntry shape
frontend/prisma/schema.prisma                            + 1 enum, 1 model (appended)
frontend/services/agent-framework/runtime/agent-runtime.ts   reservation-charge before the executor; reconci
frontend/scripts/validate-agent-{runtime,supervisor,integrity,authorization}.ts   + AGENT_CREDIT_INMEMORY=1
frontend/package.json                                    + "validate:agent-credit"
```

Not touched: `services/billing/*` / `EntitlementService` / `aiMessages` entitlement (pool-unification is a deliberate follow-on), contracts A1-A8 behaviour, legacy agents UI. No API routes. No pricing change.

---

7. Non-goals honoured

- **No pricing change** - per-tool amounts are the placeholder costs the `ToolGateway` already computes; the ledger records whatever it's given.

- No pool-unification with the `aiMessages` billing entitlement (out of scope, `services/billing/*` is off-limits per L2.5).
- No queue/worker - the resumable `tick()` model is unchanged.
- No new agents / tools / engines / UI.
- `run.creditsConsumed` is **not** the ledger - it stays as the budget counter.

---

8. Status

A9 complete. The credit ledger is a real, immutable, idempotent, auditable accounting authority. The runtime reserves credits before the executor and reconciles after; a resumed tick never double-charges; a run that can't afford its next tool call terminates `credit_limit` / `insufficient_credits`. Migration is generated + reviewed, **not applied**.

G09 review requested - schema + `migration.sql` + the ledger. On acceptance -> authorize applying the A9 migration, then A10 (Evaluation + Observability).

End AN1.11.